

Overview

This week we prepared the training and test sets saved in Week 3 (1,792 and 448 customers) for modelling. The processing ran in this order:

1. Remove duplicates
2. Clean the data
3. Remove inconsistent rows
4. Fill missing values
5. Create new features
6. Log transform
7. One-hot encode education
8. Standardize numeric features

Anything learned from data came from the training set only: the fill values, the reference date for the new features, and the scaling parameters. Rule-based fixes were applied to both sets in the same way. Steps that remove rows because of data problems were applied only to training, so the test set still reflects the data the model will meet in practice.

After processing, the training set has 1,608 customers and no missing values in either set.

1. Removing duplicates

In Week 3 we found customers who appear more than once under different IDs, some with conflicting Response labels. We handled the two sets differently:

- **Training: conflicting copies.** We removed every copy of any customer whose copies had different Response labels, as the labels contradict each other, which dropped 34 rows. This way, the model does not learn inconsistent patterns.
- **Training: remaining duplicates.** We kept one copy of each remaining duplicated customer, dropping 144 more rows. Repeated copies would otherwise give those customers extra weight during training.
- **Test.** We removed only exact copies (same features and same label), which dropped 36 rows. We left conflicting copies in place so the test set keeps its original structure.

2. Cleaning the data

We wrote these fixes as a single cleaning function and applied it to both sets:

- **Removed columns not useful for modelling.** ID is a unique identifier, and Z_CostContact and Z_Revenue have the same value for every customer, so none of the three can help predict Response.
- **Grouped marital status into a partnered flag.** Week 3 showed that Married and Together customers respond at about 10%, while Single, Divorced, and Widow customers respond at about 22%. We replaced Marital_Status with a new 0/1 column, Is_Partnered:
 - Married and Together (1)
 - Single, Divorced, Widow, and Alone (0)
 - Absurd and YOLO — missing, since they don't describe a real marital status
- This also removes the very small categories found in Week 3.
- **Merged "2n Cycle" into Master.** "2n Cycle" is a master's-level degree, so the two represent the same education level.
- **Treated implausible values as missing.** Birth years before 1920 are likely data entry errors, and the \$666,666 income is likely a placeholder rather than a real value. We set both to missing so they could be filled in Step 4.
- **Converted the enrollment date.** Dt_Customer was stored as text, so we converted it to a date to allow tenure to be calculated.

After cleaning, the training set had missing values in three columns:

- Income: 17 (the 16 originally missing plus the placeholder)
- Year_Birth: 2
- Is_Partnered: 2

3. Removing inconsistent rows (training only)

Week 3 identified two kinds of rows whose numbers contradict each other:

- customers with recorded spending but zero web, catalog, or store purchases
- customers with more deal purchases than total purchases

We dropped these rows from the training set, removing 6 rows. The test set was left unchanged.

4. Filling missing values

We filled missing values only after all row removals were complete, so the fill values come from the final training set:

- **Income and Year_Birth:** filled with the training median. The median is less affected by extreme values than the mean.
- **Is_Partnered:** filled with the most common value in training.

The same training-based values were used to fill the test set. After this step, neither set has any missing values.

5. Creating new features

We created five features through aggregation and date calculations:

- **Age:** the reference year minus birth year
- **Tenure_Days:** days between the customer's enrollment date and the reference date
- **Total_Spend:** the sum of the six spending categories
- **Total_Purchases:** web, catalog, and store purchases combined
- **Total_Prev_Accepted:** the number of the five earlier campaigns the customer accepted

The reference date is the latest enrollment date in the training set, and we used the same date for the test set. Total_Prev_Accepted was included because Week 3 showed response rising steadily with the number of earlier campaigns accepted.

Year_Birth and Dt_Customer were then dropped, since Age and Tenure_Days replace them. We chose not to combine Kidhome and Teenhome into a single Children column, because young children and teenagers at home relate to response differently.

In the training set:

- Age ranges from 18 to 74 (median 44).
- Tenure_Days ranges from 0 to 699 (median 352).
- Total_Spend ranges from 8 to 2,524 (median 411).
- Total_Purchases ranges from 1 to 32 (median 12).
- Total_Prev_Accepted ranges from 0 to 4 (median 0).

	Age	Tenure_Days	Total_Spend	Total_Purchases	Total_Prev_Accepted
count	1608.0	1608.0	1608.0	1608.0	1608.0
mean	45.3	352.5	622.9	12.7	0.3
std	11.7	201.4	609.6	7.2	0.7
min	18.0	0.0	8.0	1.0	0.0
25%	37.0	180.0	70.0	6.0	0.0
50%	44.0	352.0	411.0	12.0	0.0
75%	55.0	525.0	1079.2	19.0	0.0
max	74.0	699.0	2524.0	32.0	4.0

6. Log transforming skewed columns

Week 3 showed that the six spending columns, NumDealsPurchases, and NumCatalogPurchases are strongly right-skewed: most customers spend or buy relatively little, while a small group of high spenders forms a long tail. Week 3 also showed these high values are real customers rather than errors, so instead of removing or capping them, we applied a log transform to pull in the long tail.

We used $\log(1 + x)$, which calculates $\log(1 + x)$, instead of a standard log. Many customers have \$0 in some spending categories or zero deal or catalog purchases, and the log of zero is undefined. So with $\log(1 + x)$, zeros stay zero, and the transform still compresses large values.

The transform is a fixed formula with nothing learned from the data, so we applied it to both the training and test sets in the same way. The skew values below were measured on the processed training set. They differ slightly from Week 3 because duplicates and inconsistent rows have since been removed.

The transform brought every column close to symmetric:

- Spending columns: skew fell from between 1.13 and 2.11 to between -0.58 and 0.05 . The largest change was for meat products (2.11 to -0.12).
- NumCatalogPurchases: skew fell from 2.08 to 0.11.
- NumDealsPurchases: skew fell from 2.29 to 0.68. This is still mildly skewed but far less than before.

Income was not transformed. Once the \$666,666 placeholder was set to missing during cleaning, Income's skew was only 0.29. A log transform would push it to -1.13 , making the distribution more skewed in the opposite direction.

This step comes after feature creation and before standardization. Total_Spend was therefore calculated from the original dollar amounts, and the transformed columns are then standardized in Step 8.

7. One-hot encoding education

We converted Education into separate 0/1 columns, so the model doesn't assume equal gaps between degree levels. After the "2n Cycle" merge, four levels remained. Basic was dropped to serve as the baseline, leaving three columns: Edu_Graduation, Edu_Master, and Edu_PhD.

The test set was then aligned to have exactly the same columns as the training set.

8. Standardizing numeric features

We standardized all non-binary columns to a mean of 0 and a standard deviation of 1, so that features measured in different units (dollars, counts, days, years) are on a comparable scale. The 20 scaled columns are:

- Income, Recency, Age, and Tenure_Days
- Kidhome and Teenhome
- the six spending columns
- the five purchase and web-visit columns
- Total_Spend, Total_Purchases, and Total_Prev_Accepted

Columns that only take the values 0 and 1 were left unscaled: the five earlier-campaign flags, Complain, Is_Partnered, the three education columns, and the target, Response.

The scaler was fit on the training set and then applied to the test set using those same parameters. After scaling, every scaled training column has a mean of 0.00 and a standard deviation of 1.00.